MODULE 14 · DOMAIN 1 - DEVELOPMENT WITH AWS SERVICES

Containers, Beanstalk and the Rest of Compute

ECS, Fargate, App Runner, Elastic Beanstalk and EC2 – and the clause that picks one when several would work.

DOMAIN 1 of 4 EXAM WEIGHT 32% MODULE 14 of 34

admin@sthithapragna.com

AWS compute is a spectrum of **how much you manage**. Every question is asking where on that spectrum the scenario belongs.

The compute spectrum

SERVICE	YOU PROVIDE, AND YOU MANAGE
AWS Lambda	A function. Nothing else. 15-minute ceiling
AWS App Runner	A container image or source repo, and a port
AWS Fargate	A container and a task definition. No instances
Amazon ECS on EC2	Containers plus the instances they run on
Amazon EKS	The same, but Kubernetes. Choose it when Kubernetes is required
Elastic Beanstalk	Application code. It provisions and manages EC2 for you
Amazon EC2	Everything. Full control, full responsibility

ON THE EXAM

Move down the table only when a stated requirement forces you to. 'Least operational overhead' means move up it.

Amazon ECR

- A private container registry. Authentication is **IAM**, via a token from GetAuthorizationToken.
- **Image scanning**: basic on push, or enhanced scanning powered by Amazon Inspector.
- **Lifecycle policies** expire old images automatically, which is how you control storage cost.
- **Tag immutability** stops a tag being reused, so **v1.2** always means the same image.
- Cross-account access needs a repository policy, which is a resource-based policy.

> ON THE EXAM
>
> Tag immutability is worth knowing. Without it, redeploying 'latest' gives you a different image than the one you tested, and nothing records that.

The four ECS objects

- A **task definition** is the blueprint: image, CPU, memory, ports, environment, roles.
- A **task** is one running instantiation of that blueprint.
- A **service** keeps a desired number of tasks running and registers them with a load balancer.
- A **cluster** is the logical grouping the tasks run in.
- Task definitions are **versioned**. Each change creates a new revision, and you deploy a revision.

> ON THE EXAM
>
> Run a task for a one-off job. Create a service for something that must stay running. That distinction appears directly in scenarios.

Fargate or EC2 launch type?

FARGATE

No instances at all

- Pay per task, per second
- No patching or capacity planning
- Each task gets its own kernel
- No GPU, no privileged containers
- Dearer per unit of raw compute

EC2 LAUNCH TYPE

You run the instances

- Pay for instances, used or not
- You patch and scale the fleet
- Bin-packing gives better density
- GPUs, Spot, custom AMIs available
- Cheaper at steady high utilisation

THE TIEBREAKER

Fargate for variable load and least operational overhead. EC2 launch type for steady high utilisation, GPUs, or anything needing host-level control.

Task role or task execution role?

TASK ROLE

What your code may do

- Used by the application in the container
- Reads S3, writes DynamoDB, calls SQS
- Credentials reach the container at runtime
- Analogous to a Lambda execution role
- Optional, if your code calls nothing

TASK EXECUTION ROLE

What ECS may do to start it

- Used by the ECS and Fargate agents
- Pulls the image from ECR
- Sends container logs to CloudWatch
- Fetches secrets named in the task definition
- Not accessible to your code

THE TIEBREAKER

The clean test: a failure before the container starts is the execution role. A failure inside your application code is the task role.

Networking and secrets

- **awsvpc** network mode gives each task its own elastic network interface and private IP.
- That means security groups apply **per task**, which is why Fargate requires this mode.
- A service registers tasks with an **ALB target group**, which handles dynamic ports and health checks.
- The **secrets** field injects Secrets Manager or Parameter Store values as environment variables.
- Those secrets are fetched by the **task execution role**, never by your application.

> ON THE EXAM
>
> The secrets field is the right answer for credentials in containers. Putting a password in the environment block of a task definition exposes it to anyone who can describe the task.

ECS deployment types

- **Rolling update** is the default. ECS replaces tasks gradually within the service.
- **minimumHealthyPercent** and **maximumPercent** control how far capacity may dip or grow.
- **Blue/green via CodeDeploy** stands up a whole new task set behind a second target group.
- That supports canary and linear traffic shifting, plus automatic rollback on a CloudWatch alarm.
- **External** hands control to a third-party deployment controller.

> ON THE EXAM
>
> Rolling update cannot roll back instantly, because the old tasks are gone. Blue/green keeps them until you shift traffic, which is what makes rollback immediate.

EKS and App Runner

- **Amazon EKS** is managed Kubernetes. Choose it when **Kubernetes specifically** is required.
- It carries real operational weight. If the scenario does not name Kubernetes, it is probably not EKS.
- **AWS App Runner** takes a container image or a source repository and runs it as a web service.
- It handles load balancing, TLS, autoscaling and deployments with no configuration.
- HTTP request-response services only. No batch jobs, no queue workers.

> ON THE EXAM
>
> App Runner is the answer when a small team wants a containerised web service running with the least possible effort and does not want to learn ECS.

Elastic Beanstalk

- You upload application code. Beanstalk provisions EC2, an ASG, a load balancer and CloudWatch.
- **The resources are yours.** You pay for the EC2 instances. Beanstalk itself is free.
- An **application** holds **versions**, deployed into **environments** such as dev and prod.
- **.ebextensions** YAML files in the source bundle customise the environment.
- Blue/green is done by deploying to a second environment and **swapping the CNAMEs**.

> ON THE EXAM
>
> Beanstalk is a convenience layer over EC2, not a serverless service. If a scenario stresses no servers or no idle cost, Beanstalk is the wrong answer.

Beanstalk deployment policies

POLICY	BEHAVIOUR
All at once	Every instance at once. Fastest, with downtime
Rolling	In batches. No downtime, reduced capacity
Rolling + additional batch	Extra batch first, so capacity is maintained
Immutable	New instances, then swapped. Safest, doubles the fleet
Traffic splitting	Immutable, then a percentage of traffic. Canary

ON THE EXAM

Rollback speed is the discriminator. Immutable and traffic splitting leave the old instances untouched.

Currency note: Amazon Linux 2 has retired

- **Amazon Linux 2 reached end of life on 30 June 2026.** All AL2-based Beanstalk platform branches retired with it.
- ECS-optimised Amazon Linux 2 AMIs ended support on the same date.
- From **30 September 2026**, retired branches cannot be used for new environments.
- After **15 January 2027**, their default AMIs become inaccessible, so autoscaling and instance replacement fail.
- The migration target is **Amazon Linux 2023**. AL2023 enforces **IMDSv2** by default.

> ON THE EXAM
>
> This does not change an exam answer, but it is the kind of dated fact a scenario can rest on. The IMDSv2 default is the part that breaks older bootstrap scripts.

EC2 for developers

- **User data** is a script run once at first boot, as root. It bootstraps the instance.
- An **instance profile** attaches an IAM role, so the SDK gets credentials automatically.
- **Never put access keys on an instance.** An instance profile is always the correct answer.
- **Instance metadata** at 169.254.169.254 exposes instance details and role credentials.
- **IMDSv2** requires a session token first, which defeats the SSRF attacks IMDSv1 allowed.

> ON THE EXAM
>
> The credential chain from module 2 ends here: environment variables, then shared config, then the instance profile via metadata. That last hop is what IMDSv2 protects.

Choosing compute

THE SCENARIO SAYS	THE INTENDED ANSWER
Event-driven, short, spiky, no idle cost	AWS Lambda
Longer than 15 minutes, or always running	Fargate or ECS
Containerised web service, smallest possible effort	AWS App Runner
Containers, no instances to manage	ECS on Fargate
GPUs, custom AMIs, or steady high utilisation	ECS on EC2
Existing Kubernetes manifests or tooling	Amazon EKS
Just deploy my code, do not make me choose	Elastic Beanstalk

ON THE EXAM

The 15-minute Lambda ceiling is the single most common discriminator on this table, and it decides scenario 2 shortly.

Six scenarios. For each, decide where on the spectrum before you read the options.

A new ECS service on Fargate fails to start. The task status goes to STOPPED almost immediately with the message CannotPullContainerError, and no application logs ever appear in CloudWatch.

The container image is in a private ECR repository in the same account. The task definition has a task role granting DynamoDB access, which the application needs once running.

What is the most likely cause?

A The task role is missing DynamoDB permissions

B The ECR repository policy is blocking the account

C The task execution role is missing or lacks ECR pull and CloudWatch Logs permissions

D The container is exiting because of an application error

◆ PAUSE AND DECIDE

What is the most likely cause?

A Task role missing DynamoDB

The task role is used by application code, which never ran. It also could not cause a pull failure.

B Repository policy blocking

Possible in principle, but same-account access is granted by the IAM identity. A repository policy is for cross-account, which this is not.

C Task execution role

Pulling from ECR and writing to CloudWatch Logs are both execution role duties. Missing permissions explain the pull failure and the absent logs together.

D Application error

The image was never pulled, so the application never started. An application error would produce logs and a different stop reason.

THE SIGNAL

Two symptoms, one cause: the image could not be pulled AND no logs appeared. Both are execution role responsibilities, which is what makes it a single explanation.

A nightly job ingests a 40 GB data extract, validates it, and loads it into a database. The process takes between 40 and 90 minutes depending on file size. It currently runs on a Lambda function that fails partway through most nights.

The team wants the least operational overhead possible and does not want to manage servers or patch anything.

What should the job run on?

A An ECS task on Fargate, triggered by an EventBridge schedule

B A Lambda function with memory raised to 10,240 MB

C A Step Functions workflow that chains several Lambda functions

D An EC2 instance started and stopped by a scheduled script

◆ PAUSE AND DECIDE

What should the job run on?

A Fargate task on a schedule

No 15-minute ceiling, no servers to manage, and it runs only when scheduled. It satisfies both the duration and the operational overhead requirement.

B Maximum memory Lambda

More memory makes it faster, not longer. 15 minutes is a hard limit that no configuration changes.

C Step Functions chaining Lambdas

Genuinely viable if the work splits cleanly, and it requires rewriting a single sequential process into coordinated chunks. Far more effort than the scenario allows.

D Scheduled EC2 instance

Works, and hands back patching, AMI management and the scheduling script. That is the operational overhead the team said it did not want.

THE SIGNAL

'40 to 90 minutes' is the whole question. The 15-minute Lambda ceiling is hard, and Fargate is the least-managed thing on the other side of it.

A Beanstalk environment runs a customer-facing API on eight instances. Deployments must never reduce capacity, must never serve a mix of old and new versions to different users, and must be reversible within seconds if error rates rise.

The team accepts a longer deployment time and a temporarily larger fleet in exchange for that safety.

Which deployment policy should be configured?

A All at once

B Rolling

C Rolling with additional batch

D Immutable

◆ PAUSE AND DECIDE

Which deployment policy should be configured?

A All at once

Takes the whole environment down during the deploy. It fails the capacity requirement outright.

B Rolling

Reduces capacity while batches update, and serves a mix of versions during the deploy. Fails two requirements.

C Rolling with additional batch

The closest wrong answer. It maintains capacity, but still serves a mix of versions mid-deploy, and rollback means another rolling deploy backwards.

D Immutable

New instances in a separate ASG, swapped only when healthy. Capacity is maintained, no mixing, and rollback is terminating the new instances - which is instant.

THE SIGNAL

Three requirements, and 'reversible within seconds' is the one that eliminates C. Rolling deployments have already replaced the old instances; immutable has not.

A three-person startup has a containerised Node.js web API in a Dockerfile. They need it publicly reachable over HTTPS with a certificate, scaling with traffic, and redeploying whenever they push to their main branch.

Nobody on the team knows ECS or Kubernetes, and they explicitly do not want to learn either. There are no batch jobs or background workers.

What should they deploy to?

A. An ECS service on Fargate behind an Application Load Balancer

B. AWS App Runner, pointed at the source repository

C. An EKS cluster with a Kubernetes Deployment and Ingress

D. An Elastic Beanstalk multi-container Docker environment

◆ PAUSE AND DECIDE

What should they deploy to?

A ECS on Fargate with an ALB

Entirely correct, and it means learning task definitions, services, target groups and listeners. The scenario rules that learning out.

B App Runner from the source repo

Builds from the repository, provides HTTPS and a certificate, autoscales, and redeploys on push. Every requirement, with essentially no configuration.

C EKS

The most operational overhead of any option, for a team that named Kubernetes as something they will not learn.

D Beanstalk Docker environment

Plausible, and it provisions EC2 instances the team then owns and patches, which is more than App Runner asks of them.

THE SIGNAL

This is an HTTP request-response service with no background work, which is exactly App Runner's boundary. Change 'no batch jobs' to 'plus a queue worker' and the answer becomes ECS.

> An ECS service on Fargate serves a payment API. Deployments currently use a rolling update. After a recent bad release, error rates rose and the team took eleven minutes to restore service by redeploying the previous task definition.
>
> They want a new version validated against real traffic gradually, and want reverting to be immediate rather than another deployment.

How should deployments be changed?

A. Keep rolling updates but set minimumHealthyPercent to 100

B. Deploy to a second ECS cluster and switch Route 53 records

C. Use the blue/green deployment type with CodeDeploy, shifting traffic in canary steps with an alarm-triggered rollback

D. Add a Lambda function that monitors errors and redeploys the old task definition

◆ PAUSE AND DECIDE

How should deployments be changed?

A minimumHealthyPercent 100

Maintains capacity during the deploy and does nothing for rollback. The old tasks are still replaced, so reverting is still a redeployment.

B Second cluster plus Route 53

Achieves blue/green by hand, at the cost of a duplicate cluster and DNS TTL delays that make switchover far from immediate.

C Blue/green with CodeDeploy

The new task set runs alongside the old one, traffic shifts in canary steps, and rollback is shifting the listener back - seconds, with no redeployment.

D Lambda redeploy on error

Automates the eleven-minute recovery rather than shortening it. A redeployment is still a redeployment.

THE SIGNAL

'Immediate rather than another deployment' rules out every option that has already replaced the old tasks. Only blue/green keeps them running.

An image thumbnailing service runs as an always-on ECS service on Fargate with two tasks, costing about 700 dollars a month. Traffic analysis shows it is idle 94% of the time, handling short bursts when users upload photos.

Each thumbnail takes under three seconds. The code is a small Node.js function in a container, and the team is willing to change how it is packaged.

What change would most reduce cost?

A Move the code to a Lambda function triggered by S3 event notifications

B Reduce the ECS service to one task

C Move the service to ECS on EC2 using Spot Instances

D Add ECS service auto scaling based on CPU utilisation

◆ PAUSE AND DECIDE

What change would most reduce cost?

A Lambda on S3 events

Three-second bursts with 94% idle is precisely Lambda's shape. You pay only per invocation, so the idle 94% costs nothing at all.

B One task

Halves the bill and keeps paying for 94% idle. It also removes the redundancy of a second task.

C ECS on EC2 with Spot

Cheaper per hour, and still billed by the hour for an idle service. It also adds instances to manage, moving down the spectrum.

D Service auto scaling

Fargate services scale to a minimum task count, not to zero. There is always at least one task running and billed.

THE SIGNAL

This is the deliberate reverse of scenario 2. There, duration pushed the work off Lambda; here, idle time pulls it back on. Both turn on the same spectrum.

What to carry forward

- **Task role is your code. Task execution role is ECS starting the container.** Failure before start is the execution role.
- **The 15-minute Lambda ceiling** is the most common compute discriminator on this exam.
- Rolling replaces old instances, so rollback is another deploy. **Immutable and blue/green keep them.**
- App Runner for an HTTP service with least effort. EKS only when Kubernetes is named.
- ECS services scale to a minimum task count. **Only Lambda scales to zero.**

> ON THE EXAM
>
> That completes the Domain 1 services. Next: module 15 opens Domain 2 with IAM policies and how access is actually decided.